These action methods currently have a lot of code to them, which hides the intent of each action. There's also quite a lot of duplication between the methods, such as checking that the `Recipe` entity exists and formatting exceptions.

In this section you're going to refactor this controller to use filters for all the code in the methods that's unrelated to the intent of each action. By the end of the chapter you'll have a much simpler controller that's far easier to understand, as shown here.

**Listing 22.2 Recipe web API controller after refactoring to use filters**

```
[Route("api/recipe")]
[ValidateModel]                                          ❶
[HandleException]                                        ❶
[FeatureEnabled(IsEnabled = true)]                       ❶
public class RecipeApiController : ControllerBase
{
    public RecipeService _service;
    public RecipeApiController(RecipeService service)
    {
        _service = service;
    }

    [HttpGet("{id}")]
    [EnsureRecipeExists]                                 ❷
    [AddLastModifiedHeader]                              ❷
    public IActionResult Get(int id)
    {
        var detail = _service.GetRecipeDetail(id);       ❸
        return Ok(detail);                               ❸
    }

    [HttpPost("{id}")]
    [EnsureRecipeExists]                                 ❹
    public IActionResult Edit(
        int id, [FromBody] UpdateRecipeCommand command)
    {
        _service.UpdateRecipe(command);                  ❺
        return Ok();                                     ❺
    }
}
```